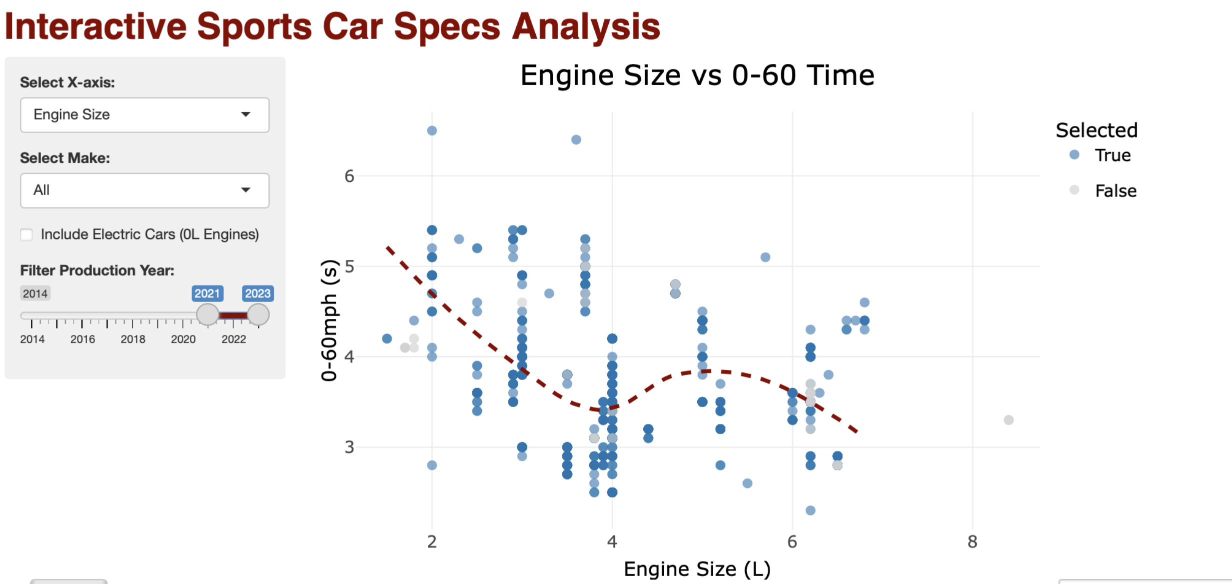
STAT 436 HW2

Nate Colburn

[Shiny App](#)
[Code](#)



Show 25 entries

Search:

Make	Model	Year	Engine Size	HP	Torque	0-60
Porsche	911	2022	3.0	379	331	4.0
Lamborghini	Huracan	2021	5.2	630	443	2.8
Ferrari	488 GTB	2022	3.9	661	561	3.0
Audi	R8	2022	5.2	562	406	3.2
McLaren	720S	2021	4.0	710	568	2.7
BMW	M8	2022	4.4	617	553	3.1
Mercedes-Benz	AMG GT	2021	4.0	523	494	3.8
Chevrolet	Corvette	2021	6.2	490	465	2.8

Introduction

As a sports car enthusiast, I wanted to answer the question of "How do different car specs, particularly engine size, affect acceleration (0-60mph time)"? I found a [dataset on Kaggle](#) that provides information such as Make, Model, Price, Horsepower, 0-60 time, and more on 900+ sports cars produced between 2014 and 2023 by a variety of different manufacturers. I find this data quite interesting, as it allows us to look at a wide range of sports cars and see both how they stack up against each other and how their design influences their performance. Most importantly, the results ended up surprising me.

Data Preparation and Interface Decisions

- I started by preparing the cars dataset. First, I cleaned up the numeric data (Price, HP, 0-60, Torque) removing unwanted characters. I filtered out all cars with a price above or equal to $1,000,000 or greater than or equal to 10,000hp (there were a few cars with incorrect horsepower data that was far too high). I also set electric car engine sizes to 0L, so that they could be viewed on the engine size plot.
- A scatterplot with tooltips made the most sense for my visualization so one could examine individual cars while also observing overall trends. I think the dark red and blue works well as a sports car aesthetic. I also added the table below to view the entirety of the data.
- I decided on making the x-axis modifiable because I wanted to show how different variables influenced the acceleration, however, I didn't want to facet the graphs, as then each point would be too small to examine individually. I wanted to be able to filter between makes so users could look at their favorite car brands, and adding a year slider made sense so that one could examine the improvement of a specific car over the years. Lastly, I added a checkbox for electric cars because they don't have an engine (I set them to 0L), so I wanted users to be able to toggle this on the engine size plot.

Interesting/Unexpected Findings

- I expected all of the x-axis variables (Engine Size, Price, Torque, and HP) to reduce 0-60 time as they rose, which was for the most part true, however, Engine Size surprised me. The cars with the quickest acceleration mostly had 4L engines, while the car with the largest engine, 8.4L, was not very quick. I was expecting that the greater the engine size, the quicker the car, but I was wrong.
- My visualization helped me make a cool discovery. I'm a big car fanatic and I had heard of the Ultima RS, but I was unaware of the incredible performance of the car for its price. While it costs $220,000, it outperforms the Ferrari SF90 Stradale, a $625,000 car.
- I was also surprised by the amount of clusters on the graph, but I quickly realized that this was due to multiple model years of the car being plotted. This discovery was the inspiration for the year slider.

Reactive Graph Structure

- The reactive graph structure of my app is designed to dynamically update the dotplot and data table based on the filters selected by the user. I have three input nodes, or dynamic queries, for the x-axis, make, and year filters, as well as a checkbox for including electric cars. These inputs are connected to the current_data() reactive expression, which is recalculated whenever an input changes so that the visualization is updated live.
- The current_data() reactive expression acts as a central node that processes the filters and produces the updated dataset. From current_data(), there are edges leading to the dotplot and the data table outputs. These outputs are updated dynamically whenever the

current_data() expression is modified, reflecting the filtered data, and the user can quickly view changes in the results.